

File Edit View Terminal Help											
top 00:50:04 up 110, 0 users, load average: 0.09, 0.05, 0.01											
Tasks: 218 total, 6 running, 212 sleeping, 0 stopped, 0 zombie											
Cpu0 : 36.34us, 11.96s, 0.00ni, 51.06id, 0.0wa, 0.0hi, 0.0si, 0.0st											
Mem: 4059396k total, 173532k used, 232464k free, 6456k buffers											
Swap: 7815580k total, 0k used, 7815580k free, 593424k cached											
PID	USER	PR	NI	VIRT	RES	SHR	S	%CPU	%MEM	TIME+	COMMAND
3112	richuss	20	0	217k	14k	9m	R	43	0.4	1:52.33	gnome-terminal
1322	root	20	0	535m	79m	13m	R	15	2.0	5:27.62	Xorg
2419	richuss	20	0	768m	27m	27m	S	9	1.9	13:58.27	exe
3420	richuss	20	0	48936	4088	740	S	7	0.1	0:00.87	python
1802	richuss	20	0	675m	94k	35m	S	5	2.4	4:06.07	firefox
3417	richuss	20	0	48936	4036	708	R	4	0.1	0:00.64	python
3427	richuss	20	0	48936	4036	708	S	4	0.1	0:00.77	python
3430	richuss	20	0	48936	4088	740	S	3	0.1	0:00.52	python
1778	richuss	20	0	253m	34k	7676	S	3	0.9	1:09.62	cmpriz.real
1794	richuss	20	0	592m	138m	35m	R	2	3.5	4:32.08	chrome
3421	richuss	20	0	48936	4036	708	S	2	0.1	0:00.17	python
3419	richuss	20	0	48936	4036	708	S	1	0.1	0:00.76	python
3420	richuss	20	0	48936	4036	708	S	1	0.1	0:00.85	python
1661	richuss	20	0	265m	514k	3640	S	1	0.1	1:21.67	pulseaudio
2076	richuss	21	1	853m	34k	15m	S	1	0.9	0:25.53	chrome
3882	richuss	21	1	776m	17m	10m	S	1	0.4	0:07.09	chrome

Figure 1: Multiprocessing sub-processes running in parallel

see the different outputs generated by the different sub-processes that are created.

```
#!/usr/bin/env python2.7

from multiprocessing import Process

def test(name):
    print 'welcome ' + name + ' to multiprocessing!'

if __main__ == __name__:
    p1 = Process(target = test, args = ('Rich',))
    p2 = Process(target = test, args = ('Nus',))
    p3 = Process(target = test, args = ('Geeks',))

    p2.start()
    p1.start()
    p3.start()

    p1.join()
    p2.join()
    p3.join()
```

To verify that you have multiple sub-processes, run *waste*. *py* and *counting.py* (shown below) and see how different sub-processes print to the console in a jumbled manner. The *counting.py* script has a delay after printing, so you can see that the sub-processes' output is slower. I also captured the *top* command's output (Figure 1) while running *waste.py* on my dual-core machine; it clearly shows that multiple Python sub-processes are running. It also shows the balanced distribution of the sub-processes across multiple cores, in the form of the cores' idle percentage.

```
#!/usr/bin/env python2.7
# === counting.py ===
from time import sleep
from multiprocessing import Process
```

File Edit View Terminal Help									
richuss@geekbar-laptop:~/Development/Package/Works/process-0.5/example\$ python simple.py									
0 1 2 3 4 5 6 7 8 9									
10 11 12 13 14 15 16 17 18 19									
20 21 22 23 24 25 26 27 28 29									
30 31 32 33 34 35 36 37 38 39									
40 41 42 43 44 45 46 47 48 49									
50 51 52 53 54 55 56 57 58 59									
60 61 62 63 64 65 66 67 68 69									
70 71 72 73 74 75 76 77 78 79									
80 81 82 83 84 85 86 87 88 89									
90 91 92 93 94 95 96 97 98 99									
Time taken: 100.04121268s									
richuss@geekbar-laptop:~/Development/Package/Works/process-0.5/example\$ python pmap.py									
0 1 2 3 4 5 6 7 8 9									
10 11 12 13 14 15 16 17 18 19									
20 21 22 23 24 25 26 27 28 29									
30 31 32 33 34 35 36 37 38 39									
40 41 42 43 44 45 46 47 48 49									
50 51 52 53 54 55 56 57 58 59									
60 61 62 63 64 65 66 67 68 69									
70 71 72 73 74 75 76 77 78 79									
80 81 82 83 84 85 86 87 88 89									
90 91 92 93 94 95 96 97 98 99									
Time taken: 10.046111271s									
richuss@geekbar-laptop:~/Development/Package/Works/process-0.5/example\$									

Figure 2: Performance boost through process pmap

```
def count(number, label):
    for i in xrange(number):
        print ' ' + str(i) + ' ' + label
        sleep((number - 15)/10.0)

if __main__ == __name__:

    p1 = Process(target = count, args = (20, ' Rich',))
    p2 = Process(target = count, args = (25, ' Nus',))
    p3 = Process(target = count, args = (30, ' Geeks',))

    p1.start()
    p2.start()
    p3.start()

    #!/usr/bin/env python2.7
    # === waste.py ===
    from multiprocessing import Process

    def waste(id):
        while 1:
            print str(id) + ' Total waste of CPU cycles!'

    if __main__ == __name__:

        for i in xrange(20):
            Process(target = waste, args = (i,)).start()
```

If you need the thread kind of locking, then *multiprocessing* provides the *Lock* class too; you can use the *acquire* and *release* methods of the *Lock* object. There is also a *Queue* class and a *Pipe* function, for communication between sub-processes. The *Queue* is both thread- and process-safe, so you don't have to worry about locking and unlocking while working with it. The *Pipe()* returns a pair of connection objects that are connected by a pipe, in two-way mode, by default. If you want a pool of worker sub-processes, then you can use the *Pool* class, which offloads work to created sub-processes in a few different ways. Run *miscellaneous.py* (shown below) to see some of these concepts in action.